Monolith Engine — System & Project Documentation

Architecture, Features, Functional Modules, Workflows, Current Implementation and Pending Scope

Package name in repository: `monolith-engine`. Analyzed as a Next.js/Prisma/PostgreSQL enterprise resource-planning (ERP) system for shipping, logistics (Customs House Agent operations), accounting, HR, CRM, and internal communication.

Analysis Scope - Repository: `Adarsh-Shipping-and-Services-Management-Software` (local working copy) - Branch analyzed: `main` - Latest commit at time of analysis: `9da9d75` — "Add view/archive/delete actions to leave policies table" - Total commits in history: 309 - Date of analysis: 2026-08-20 - Applications/packages included: the Next.js web application (`src/`), Prisma schema and migrations (`prisma/`), operational scripts (`scripts/`), and the Android companion app manifest (`mobile/sales-crm-android`, not inspected in depth — see Documentation Confidence section)

---

2. Document Purpose

This document exists to give the client a complete, evidence-based understanding of the Monolith Engine platform as it stands today. It explains what has been built, how the system is architected, what a user can actually do with it right now, and — separately — what remains incomplete, disabled, or only partially wired up. It is written from direct inspection of the source code, database schema, configuration, scripts, documentation, and git history, not from assumptions about what an ERP "usually" contains. Where the codebase does not provide enough evidence to support a claim, this document says so explicitly rather than guessing.

---

3. Executive Summary

Monolith Engine is a large, custom-built, in-house Enterprise Resource Planning (ERP) system for a shipping and logistics services company. It replaces or supersedes an earlier, less structured application (referred to in the codebase as the "old UI" / pre-Monolith system) and consolidates a wide range of business functions into one platform: accounting and financial reporting, customer relationship management (CRM), human resources and payroll, customs house agent (CHA) job processing and statutory filing, freight forwarding documentation, internal team communication (chat, email, shared drive, meetings), performance appraisals, learning management, and a self-service customer portal.

The system is substantial: 415 database models, roughly 230 backend API endpoints across 26 functional areas, and around 290 front-end pages. It is under active, continuous development, with 309 recorded commits and clear evidence of large, deliberately-phased engineering initiatives rather than ad hoc feature dumps.

What is solid today: Accounting, CRM, HR/HRMS, CHA job processing, freight-forwarding documentation, and internal communication (chat, email, shared drive, calendar — all backed by real Google Workspace integration) are functionally deep and actively used code paths, not scaffolding. A recently completed, security-hardened Leave Management module (22 iterative closure passes, addressing real bugs including an approval-bypass vulnerability) is a good example of the platform's engineering maturity when a module reaches completion.

What is unfinished: The platform is simultaneously in the middle of two large, overlapping migrations — a UI/design-system rewrite (moving away from an older visual system) and an accounting-module data-model rewrite (a "Phase 0 through Phase 9" effort). Both are extensively documented and tracked, but neither is finished: the UI migration's own audit tooling reports 55 of 341 routes as non-compliant with the new design system, and the accounting rewrite still carries two parallel sets of database tables (an old set and a new set) rather than one. Several specific features are explicitly marked "coming soon" in the UI itself (HR onboarding checklists, an attendance timesheets view, one CHA settings panel, two CRM

quote features). Test coverage is very uneven — Accounting and Leave Management are well tested, while fourteen of twenty-five functional modules have no automated tests at all.

Bottom line for stakeholders: this is not a prototype — it is a working, feature-rich system handling real business processes across shipping, finance, HR, and customer communication. The main risk areas are the two in-flight migrations (which create temporary complexity and duplicate code paths until finished) and uneven test/verification coverage in less-mature modules.

---

4. Project Overview

Purpose: Monolith Engine is the operational backbone for a shipping and services company, most notably one that acts as a Customs House Agent (CHA) — a licensed intermediary that files import/export customs declarations on behalf of importers/exporters in India. The platform ties together the full lifecycle of a shipping job: lead generation and quoting (CRM), customs filing and compliance (CHA), house/master bill-of-lading documentation (freight forwarding), the accounting entries that result (invoicing, payments, GST/tax settlement), the internal HR operations to run the company (attendance, leave, payroll, appraisals), and the communication layer that ties staff, customers, and job-specific collaboration together (chat, email, shared drives, meetings).

Primary users/actors (inferred from role/permission structures and route names — see §8): - Internal staff across departments: CHA operations, accounting/finance, HR, CRM/sales, and platform administrators. - External customers, via a dedicated Customer Portal for document submission, checklist responses, and query threads. - External job applicants, via a public-facing recruitment/job-seeker portal (feature-flagged). - A mobile Android companion app for CRM and HR self-service (attendance check-in/out, on-duty, tracking).

Primary business use cases: - Manage a CHA job from customer enquiry through customs clearance and billing. - Produce statutory customs filings (import/export declarations, IGM, container/invoice/item-level filing data) referencing Indian customs reference data (tariff codes, cess, RODTEP/ROSCTL/drawback schemes). - Run double-entry accounting: sales/purchase invoices, journal entries, bank reconciliation, fixed-asset depreciation, GST/tax settlement, financial statements. - Manage the employee lifecycle: recruitment, onboarding, attendance/biometric sync, leave, payroll-adjacent letters, performance appraisals, training. - Run CRM: leads, deals, quotes, service enquiries (customs clearance / freight forwarding), support tickets, and call-center QA (recording/transcription/review). - Provide integrated internal communication (Google Chat, Gmail, Drive, Calendar) directly inside the ERP, including per-job collaboration spaces. - Give external customers self-service visibility into their shipment documents and queries.

General system behavior: a single Next.js web application serves both the UI and the API; a single PostgreSQL database (via Prisma ORM) is the system of record; authentication is centralized (NextAuth with a custom database-backed session layer); authorization is role/permission-based but enforced per-route rather than by a single global gate (see §8 and §12).

---

5. Technology Stack

Layer	Technology	Purpose
Frontend	Next.js 16 (App Router), React 19, TypeScript 5	UI rendering, routing, server actions
Styling	Tailwind CSS v4 + a custom CSS-module design system	Visual design, in the middle of a migration from an older "Frappe-style" system to a canonical component system
UI components	Radix UI primitives, custom `src/components/ui` catalogue, Carbon/Lucide icons, Framer Motion/GSAP for motion, Three.js for 3D elements	Reusable interface building blocks

Layer	Technology	Purpose
Backend	Next.js API routes (`src/app/api/**`) and Server Actions	Business logic, request handling — no separate backend service
Database	PostgreSQL	System of record
ORM	Prisma 7 (custom output path, `pg` driver via `@prisma/adapter-pg`)	Type-safe database access, schema migrations
Authentication	NextAuth.js v5 (beta), Credentials + Google OAuth providers, custom database-backed session validation layer	Login, session lifecycle, server-side revocation
Authorization	Custom RBAC layer (`src/lib/rbac.ts`) — roles, permissions, department-scoped bundles	Feature/action-level access control
External integrations	Google Workspace (Gmail, Chat, Drive, Calendar), Google Gemini (AI assistant "Mona"), eSSL biometric devices (via SQL Server), GST public portal, OpenAI/Groq Whisper (call transcription), Justdial (lead import)	See §13
Testing	Vitest (three configurations: staging-DB-gated, unit-only, UI-component), Playwright (performance/security, leave e2e)	Automated verification
Containerization	Docker (multi-stage build: deps → builder → migrator → runner), docker-compose (Postgres + migration gate + app)	Local/staging environment, self-hosted deployment path
Hosting candidate	Vercel (`.vercelignore` present)	Alternative/likely production deployment target
Mobile	Separate native Android app (`mobile/sales-crm-android`), communicating with dedicated `/api/mobile/*` endpoints	CRM and HR self-service on mobile

---

6. System Architecture

Monolith Engine is a **modular monolith**: one Next.js codebase serves the entire UI and API surface, backed by one PostgreSQL database, rather than a microservices architecture. Internally, the codebase enforces modular boundaries through directory conventions and lint rules rather than network boundaries:

- `src/app` — routing layer only (pages and API route handlers). Route handlers are intentionally thin.
- `src/modules/<domain>` — the business-logic layer per functional domain (accounting, crm, hrms, cha, communication, leave, etc.). ESLint rules forbid modules importing from `src/app` , keeping logic decoupled from routing.
- `src/components` — a shared, canonical UI component catalogue, plus a legacy `monolith` component bucket that is in the process of being retired (see §21).
- `src/lib` — cross-cutting infrastructure: auth, database client, email, the four Google Workspace API clients, security/rate-limiting, the biometric device connector, performance instrumentation.
- `prisma/` — the single shared schema (415 models) and migration history for the whole system.

A background/asynchronous layer exists via **scheduled cron-style endpoints** (`src/app/api/cron/*` , protected by a shared-secret header rather than a user session) rather than a dedicated message queue or worker process — see §14.

External systems are integrated through dedicated client modules in `src/lib` (one per Google service, plus GST portal, biometric DB, and AI/transcription providers), all invoked directly from API routes or server actions rather than through a separate integration service.

```
flowchart TB
    subgraph Client["Clients"]
        Browser["Web Browser (Staff / Admin)"]
        Portal["Customer Portal (Browser)"]
        Mobile["Android App"]
    end
```

```
    subgraph App["Next.js Application (monolith-engine)"]
        Routes["App Router Pages<br/>src/app/(dashboard)/*"]
        API["API Route Handlers<br/>src/app/api/*  (~230 endpoints)"]
        Modules["Domain Modules<br/>src/modules/* (business logic)"]
        Auth["Auth & RBAC<br/>NextAuth + src/lib/rbac.ts"]
    end

    DB[(PostgreSQL<br/>via Prisma — 415 models)]

    subgraph External["External Systems"]
        Google["Google Workspace<br/>Gmail / Chat / Drive / Calendar"]
        Gemini["Google Gemini (Mona AI)"]
        ESSL["eSSL Biometric Devices (SQL Server)"]
        GST["GST Public Portal"]
        Whisper["OpenAI / Groq Whisper (call transcription)"]
        Justdial["Justdial (lead import)"]
    end

    Browser --> Routes
    Portal --> Routes
    Mobile --> API
    Routes --> API
    API --> Auth
    API --> Modules
    Modules --> DB
    Modules --> Google
    Modules --> Gemini
    Modules --> ESSL
    Modules --> GST
    Modules --> Whisper
    Modules --> Justdial
```

---

7. Repository Structure

Path	Responsibility
`src/app/(dashboard)/*`	Authenticated application pages — one directory per functional module (accounting, crm, hrms, cha, communication, attendance, ams, freight-forwarding, lms, expense, admin, etc.)
`src/app/api/*`	~230 backend API route handlers, grouped into 26 domains
`src/app/(auth)`, `src/app/customer-portal`, `src/app/invite`, `src/app/verify`	Unauthenticated/entry-point routes: login, external customer portal, invitations, verification
`src/modules/<domain>`	Business logic per domain — services, components, types — decoupled from routing
`src/components`	Shared, canonical UI primitives and layout/navigation/forms/feedback components
`src/lib`	Cross-cutting infrastructure: auth, database client, email, Google integrations, security, biometric device connector, performance instrumentation
`src/styles`	CSS-module design system, split per module (accounting, cha-expense, communication-admin, etc.)
`src/generated/prisma`	Generated Prisma Client (checked into the repository rather than the default location)
`prisma/`	Database schema, migration history, seed scripts

Path	Responsibility
`scripts/`	~96 operational scripts: accounting migration/rollout tooling, UI-migration audits, per-module verification suites, staging environment tooling, seed/bootstrap data scripts
`docs/`	UI design-system migration status/audit documents, leave-management project documentation
`mobile/sales-crm-android`	Separate native Android companion app (excluded from the web build)
`frappe_docker/design`	Residual reference material from an earlier Frappe-based system predecessor

---

8. User Roles and Access Control

Authentication: NextAuth.js v5, with two sign-in paths — email/password (bcrypt-hashed, with brute-force lockout) and Google OAuth (broad Workspace scopes for Gmail, Calendar, Chat, Drive, Directory). Google sign-in is restricted to accounts an administrator has already pre-provisioned in the system; self-service Google signup is explicitly blocked in code (`src/lib/auth.ts`).

Session handling layers a database-backed session record on top of the JWT: every request re-validates the JWT against a live `UserSession` row, enabling server-side session revocation and idle/absolute timeout independent of the JWT's own expiry. A **multi-factor authentication hook exists but is a no-op placeholder** (`verifySecondFactor()` always returns `true`) — MFA is not currently enforced despite the scaffolding being present.

Authorization: a custom RBAC system (`src/lib/rbac.ts`) built on `User → UserRole → RolePermission → Permission` tables, loaded per-user and cached for five minutes. A separate `isPlatformAdmin` boolean flag grants a platform-level bypass. The system also includes a "department-scoped permission bundle" mechanism that grants certain permission bundles automatically based on department membership (e.g., users in the Accounts department implicitly receive accounting read/full permission bundles), independent of explicit role grants — a convenience mechanism worth being aware of from a least-privilege perspective.

Enforcement pattern: there is **no single global middleware gate** over the API surface. Protection is applied per-route, converging on a small number of shared idioms — an inline session check, a shared `getSessionOrUnauth()` helper, `requirePermission()` for RBAC-sensitive routes, a shared-secret check for cron endpoints, and a separate bearer-token scheme for the mobile app. This means a route author who forgets to add the check has no framework-level backstop; it is a structural consistency risk rather than a confirmed vulnerability (see §23).

Customer Portal and the **job-seeker/recruit portal** each have entirely separate, non-NextAuth authentication flows suited to external, non-employee users.

---

9. Module-by-Module Functional Documentation

The system organizes into the following major functional modules. Each is documented in the level of depth the evidence supports; large modules (Accounting, CHA, Communication) receive fuller treatment.

Module 1 — Accounting

Purpose: Double-entry financial accounting and reporting for the organization: sales and purchase invoicing, payments, banking, fixed assets, tax, and financial statements.

Business Function: Converts operational activity (CHA jobs, freight bookings, CRM sales) into the company's books of account, and produces the reporting needed for management and statutory (GST/tax) compliance.

Main Features: sales/purchase invoices and orders, journal entries, general ledger, trial balance, balance sheet, profit & loss, bank statement import and reconciliation, recurring invoice/bill templates and scheduling, fixed-asset register and depreciation runs, tax profiles/rules/settlement, budget management, a custom report builder, transaction period locking, and a dedicated data-migration subsystem (batches/records/mappings/checkpoints/exceptions) used to move data from the legacy accounting model into the new one.

Main Components: `src/modules/accounting/*` (routes, authorization-planning, migration, rollout subfolders), ~70+ pages under `src/app/(dashboard)/accounting/*`, ~13 API route groups under `src/app/api/accounting/*`.

Database: the largest single domain in the schema (80+ `Accounting*`-prefixed models — legal entities, periods with lock requests, tax profiles, bank statement imports, reconciliation sessions, recurring schedules, financial assets/depreciation runs, budgets, a generic `AccountingDocument` / `DocumentLine` model, migration-tooling tables). Notably, **a second, older set of accounting models still exists in parallel** without the `Accounting` prefix (`Account`, `FiscalYear`, `JournalEntry`, `GeneralLedgerEntry`, `SalesInvoice`, `PurchaseInvoice`, `PaymentEntry`, `CustomerLedgerEntry`, `SupplierLedgerEntry`) — direct evidence of an in-progress rewrite that has not yet fully retired the original model set.

API Endpoints (representative):

Method	Endpoint	Purpose	Auth
GET	`/api/accounting/items`	List chart-of-items	Session + `accounting.dashboard.view` permission
—	`/api/accounting/reports/catalog`	Available report definitions	Session-gated
—	`/api/accounting/automation-rules[/[id]]`	Manage posting automation rules	Session + permission
—	`/api/accounting/custom-fields[/[id]]`	Configure custom fields	Session + permission
—	`/api/accounting/approvals/summary`	Pending-approval overview	Session-gated

Detailed Workflow: operational documents (invoices, bills, payments) are created through the accounting UI or generated by upstream modules → validated against configured tax/period rules → posted to journal/ledger entries (via a posting engine, `src/modules/accounting/posting-engine.ts`) → reflected in ledgers and financial statements → periods can be locked to prevent retroactive edits once closed.

Current Implementation Status: Implemented, with an active internal migration in progress. This is the most heavily engineered module in the system: 90+ commits across a formally phase-gated rollout (Phase 0 audit through Phase 9), 27 automated test files (the highest of any module), and extensive readiness/benchmark/safety scripts gating each phase before go-live.

Known Limitations / Pending Work: the legacy and new accounting model sets coexist (`prisma/schema.prisma`); a migration-tooling layer (batches/checkpoints/exceptions) exists specifically to reconcile this, indicating the cutover is not complete. The rounding-mode configuration has at least one explicitly unsupported value (`posting-engine.ts:532` raises a defensive runtime error rather than silently miscalculating — this is a deliberate guard, not a bug, but it does indicate the configuration surface is not fully generalized).

Evidence: `prisma/schema.prisma`; `src/modules/accounting/*`; `scripts/accounting-phase6-readiness.ts` … `scripts/accounting-phase9-*`; `docs/ui-migration-status.md`.

Module 2 — Customs House Agent (CHA) Operations

Purpose: Manage the end-to-end lifecycle of a customs clearance job — the company's core service line as a licensed CHA in India.

Business Function: From job creation through document checklists, statutory filing data (import/export declarations, IGM headers, container/invoice/item-level detail, drawback/RODTEP/ROSCTL/scheme references), approvals, and expense tracking against the job.

Main Features: job management (`jobs/[jobId]`), customer/vendor masters, a document checklist system with sections/items/approval/rework states, import/export filing headers with statutory reference data, a filing-workflow engine (see below), job-linked expense requests with payments and queries, customer advance receipts, and a due-date warning system.

Database: `ChaJob` , `ChaJobAssignment` , `ChaDocumentDefinition` / `Version` / `Exception` , `ChaChecklist` (+Section/Item/Approval/Rework), `ChaFiling` , `ChaImportFilingHeader` / `IgmHeader` / `Container` / `Invoice` / `Item` / `Declaration` , `ChaExportFilingHeader` / `Invoice` / `Item` , a family of `ChaCustoms*` tables holding external submission state and India-specific statutory master data, `ChaExpenseRequest` / `Line` / `Payment` / `Query` , `ChaCustomerAdvance` (`Receipt`).

Separately, a **generic node-graph workflow/BPM engine** exists in the schema (`FilingWorkflowTemplate` / `Version` / `Node` / `Edge` , `FilingWorkflowInstance` , `FilingNodeRun` , `FilingFieldValue` , `FilingChecklistItem` , `FilingSection49Flag` / `Extension`) — this appears to be the underlying engine that drives configurable CHA filing checklists/workflows, distinct from the fixed `ChaChecklist` model.

API Endpoints (representative): `checklist-files/[id]` , `customer-documents/[id]` , `documents/[id]` , `due-date-warnings` , `expense-artifacts/[...path]` , `jobs/create-options` , `reports/jobs/[jobId]` — 7 route groups total.

Current Implementation Status: Implemented and mature. ~70+ front-end pages, real document handling (with an explicit, clearly-labeled dev/mock fallback for document *preview* when real file bytes aren't available — not a sign of incomplete core functionality, see §22). 8 automated test files.

Known Limitations / Pending Work: one settings sub-panel is explicitly marked "Coming soon" (`cha/settings/settings-form.tsx:1101`). Git history shows a "customs filing phases" feature branch that was added and later explicitly removed/descoped (`8d65f6c` — "remove customs filing and continue ui migration"), indicating some filing-phase scope was deliberately deferred rather than shipped.

Evidence: `prisma/schema.prisma` (Cha* models); `src/modules/cha/*` ; `src/app/(dashboard)/cha/*` ; `src/app/api/cha/*` .

Module 3 — Freight Forwarding

Purpose: Produce and manage House Bill of Lading (HBL) and Master Bill of Lading (MBL) shipping documentation.

Main Features: booking creation by document type, HBL/MBL document generation per transaction, a quote-to-process workflow.

Database/API: tied into the CRM `CrmDeal` /quote pipeline and the accounting document model; specific dedicated freight models were not separately catalogued in this pass beyond the page/route evidence.

Current Implementation Status: Implemented (real pages for booking creation, HBL/MBL, process/settings). **Zero automated test files** for this module — a coverage gap.

Evidence: `src/app/(dashboard)/freight-forwarding/*` .

Module 4 — Customer Relationship Management (CRM)

Purpose: Manage the sales pipeline from lead through deal, quote, and service enquiry, plus post-sale support ticketing and call-center quality review.

Main Features: leads, deals, quotes (with PDF generation), enquiries, dedicated customs-clearance and freight-forwarding service-enquiry types, support tickets, sales incentive tracking, a sales inbox, and a call-center module (call attempts, recordings, transcripts, review/QA) with an automated transcription pipeline (OpenAI Whisper → Groq Whisper fallback chain, multi-language). A **Justdial lead-import integration** (headless-browser-based) automatically pulls in external leads.

Database: `CrmLead`, `CrmContact`, `CrmAccount`, `CrmDeal`, `CrmActivity`, `CrmTicket`, `CrmProduct`, `CrmInvoice` / `Item`, `CrmProject`, `CrmCallAttempt` / `CallRecording` / `CallTranscript` / `CallReview`, `CrmLeadSourceJustdialConfig` / `CrmExternalLeadSnapshot`.

Current Implementation Status: Implemented, ~70+ front-end pages. Two features are explicitly deferred: quote sharing and quote workflow preferences both show "coming soon (pending a database update)" toasts in the UI (`QuoteDetailsPage.tsx:759,792`) — a specific, evidenced case of a UI control existing ahead of its backing schema change. **Only 2 automated test files** despite the module's size — a notable coverage gap relative to its front-end footprint.

Evidence: `prisma/schema.prisma` (Crm* models); `src/modules/crm/*`; `src/app/(dashboard)/crm/*`; `src/lib/transcription.ts`; `src/modules/crm/justdial-import.service.ts`.

Module 5 — Human Resources (HRMS)

Purpose: Employee lifecycle management: records, payroll-adjacent processes, recruitment, and self-service.

Main Features: employee records, payroll and salary structure/revisions, an onboarding flow, a two-sided recruitment system (career portal for candidates + employer portal for hiring managers), HR letter generation, on-duty administration, an HR helpdesk, travel requests/expenses, and work-report submission.

Database: `EmploymentRecord`, `EmployeeHrmsProfile`, `HRCase`, `HRLetterTemplate` / `Request`, `TravelRequest` / `Expense`, and the recruitment tables under §"Recruit" below.

API Endpoints: 33 route groups under `/api/hrms/*` — the largest single API domain in the system — covering employees, files, hr-cases, incentives, invitations, leave requests/summary, letters, LMS linkage, on-duty, onboarding, performance, reimbursement, settings, tasks, team/reportees, time tracking, travel, and work reports.

Current Implementation Status: Implemented and broad, but with one explicit stub: the onboarding checklist view literally renders "Onboarding checklists are coming soon" (`src/modules/hrms/components/onboarding-view.tsx:10,16`) — the onboarding *record-keeping* exists, but the guided-checklist UX does not yet. 8 automated test files.

Evidence: `src/modules/hrms/*`; `src/app/(dashboard)/hrms/*`; `src/app/api/hrms/*`.

Module 6 — Attendance & Leave Management

Purpose: Track employee attendance (including biometric device sync), overtime, and manage the full leave lifecycle (requests, approvals, balances, compliance).

Main Features (Attendance): day-level punch tracking, shift assignment, biometric device sync (live and batch, against an eSSL eTimeTrackLite device via a direct SQL Server connection — `src/lib/essl.ts`), overtime entry, attendance regularization, LOP (loss of pay) tracking.

Main Features (Leave — the most recently completed module): leave types, balances, requests with approval-step workflows, policy versioning (with clone/archive/publish/compliance-check/compare), a ledger with adjustment support, comp-off credit (FIFO-based), leave encashment, delegation of approval authority, restricted-holiday handling, and a reconciliation/reporting layer. This module was built via a single large initial commit followed by **22 explicitly numbered "closure pass" commits**, each addressing a specific, named defect — including a transaction-ordering bug, an IDOR (insecure direct object reference) vulnerability, a self-approval bypass, a float-to-Decimal migration for financial

correctness in leave-balance math, "sandwich rule" holiday-escaping logic, multi-shift calendar handling, and outbound external calendar sync. Project documentation (`docs/leave-management/FINAL_REPORT.md` , `FINAL-CLOSURE-AUDIT.md`) confirms 10 new database tables and a test count that grew from 51 to 55 across the closure passes.

Database: `AttendancePunch` (`Event`), `Shift` / `ShiftAssignment` , `BiometricDevice` / `SyncLog` / `PunchImport` , `LeaveType` , `LeaveBalance` , `LeaveRequest` , `LeavePolicyVersion` , `LeaveLedgerEntry` , `LeaveApprovalStep` , `CompOffCredit` , `LeaveGrant` , `LeaveScheduleRun` , `Holiday` , `OTEntry` / `OtSettings` / `OtRecord` , `AttendanceRegularization` , `EmployeeLop` .

Current Implementation Status: Implemented. Leave Management specifically is the best-documented, most rigorously closed-out module in the codebase — 19 test files, an explicit gap-analysis scored 9.3/10 against a reference feature set (Zoho, per `docs/leave-management/ZOHO_FEATURE_RESEARCH.md`), and a documented security review that found and fixed two real pre-existing bugs (an approval bypass in `src/modules/hrms/service.ts` and an unaudited opening-balance seeding path).

Known Limitations / Pending Work — significant: `docs/leave-management/FINAL_REPORT.md` documents an **unresolved deployment blocker**: the generated database migration for the leave ledger/policy engine (`prisma/migrations/20260814090000_leave_management_ledger_policy_engine/`) has never been applied to a live database, because the configured database credential is write-denied against that environment. This means the module's schema changes, while built and tested, may not yet be live in the actual running system — this should be confirmed operationally before treating Leave Management as fully deployed. Separately, the front-end `attendance/timesheets` route is a literal stub ("Timesheets coming soon", `page.tsx:13`).

Evidence: `prisma/schema.prisma` ; `src/modules/leave/*` , `src/modules/attendance/*` ; `docs/leave-management/*` ; commits `0a86855` , `3708ad0` … `9da9d75` .

Module 7 — Performance & Learning (AMS / LMS)

Purpose: Employee performance appraisal cycles and workplace learning/training.

Main Features (AMS): appraisal cycles, self-assessment, reviewer ratings, management review, hike decisions, goals, skills tracking, a fixed-asset register (also under the `ams` route — appraisal/asset management appear co-located under this section), criteria/slabs configuration.

Main Features (LMS): course catalogue, enrollments, assignments, reports.

Database: `AppraisalCycle` , `Appraisal` , `AppraisalReviewer` , `SelfAssessment` , `ReviewerRating` , `ManagementReview` , `HikeDecision` , `Goal` , `Skill` / `EmployeeSkill` , `PerformanceFeedback` , `SalaryRevisionLetter` , `Course` / `CourseEnrollment` , `Survey` / `Question` / `Response` .

API: 16 route groups under `/api/ams/*` .

Current Implementation Status: Implemented. Two early commits explicitly marked appraisal/reviewer forms " (Not Completed)" during initial scaffolding (`9852081` , `aa9915d`) — historical only; current state was not found to carry an equivalent live stub marker. **Zero automated test files for `ams`** , despite it being a fully page-built module — a coverage gap, and one of the highest non-compliant-with-design-system route counts per the UI migration audit (see §21).

Evidence: `src/app/(dashboard)/ams/*` ; `src/app/api/ams/*` ; `prisma/schema.prisma` .

Module 8 — Communication

Purpose: Bring internal team communication — chat, email, shared drive, calendar/meetings, and per-job collaboration spaces — directly into the ERP, backed by real Google Workspace integration rather than a bespoke messaging system.

Main Features: - **Mail:** a full Gmail-client implementation (3,293-line component) — thread list/pagination, folder and label filtering, Gmail-style search, compose with CC/BCC/attachments, reply/reply-all/forward, draft send,

star/archive/trash, keyboard shortcuts, inline image preview, unsubscribe-header parsing, and the ability to link an email thread to a specific CHA job (across 8 fixed document categories) for job-centric record-keeping. - **Chat:** a real-time messenger (2,867-line component) with Server-Sent-Events (SSE) live updates and polling fallback, live presence (`UserPresenceState`) and typing indicators, read receipts sourced from Google Chat's actual read-state API, message CRUD/edit/reactions, per-space drafts, DM/space/job-space navigation, attachments, and a manual "sync with Google" action. - **Drive:** real Google Drive integration for per-job file browsing, with an explicit, clearly-labeled fallback UI state when a job's Drive folder has not yet been provisioned. - **Meetings:** real Google Calendar integration — event creation with attendees and auto-generated Google Meet links, upcoming-events listing. - **Job-linked provisioning:** creating a CHA job automatically provisions a dedicated Google Drive folder tree and a Google Chat space with the assigned team invited, via a bot service account (`src/lib/workspace-provisioning.ts`). - **Search, job-spaces, settings, and an "overview" analytics dashboard** (KPI tiles, activity feed across email/chat/mentions, meeting/drive coverage) are also present as substantial, real pages. - A **"Google Chat Live View"** route is explicitly labeled in its own metadata as "Experimental integration."

API Endpoints: 24 route groups under `/api/communication/*` , including several newly added (untracked in the working copy at time of analysis) routes for chat attachments, presence, typing, and read-state, and mail attachments/drafts — all following the same authentication pattern (inline session check, 401 on missing session) with the presence/typing endpoints reading/writing local Postgres state and the rest proxying to the relevant Google Workspace API.

Current Implementation Status: Implemented and functionally mature, despite being an area of very recent, heavy active development (per git status at time of analysis). This is not scaffolding — mail, chat, drive, and calendar all reach real Google APIs with substantial, production-shaped logic. Code-quality note: the main chat component carries a blanket ESLint-disable comment covering several rule categories, suggesting it was iterated on quickly under time pressure — functionally complete but a candidate for a later cleanup pass.

Known Limitations / Pending Work: per the UI-migration audit, the `/chat` route specifically has been reworked twice and remains `NON_COMPLIANT` with the new design system as of the most recent audit run. Only 2 automated test files exist for a module of this functional depth.

Evidence: `src/modules/communication/components/mail-workspace.tsx` ; `src/app/(dashboard)/communication/chat/page.tsx` ; `src/lib/google-{chat,gmail,drive,calendar}-client.ts` ; `src/lib/workspace-provisioning.ts` ; `src/app/api/communication/*` .

Module 9 — Customer Portal

Purpose: Give external customers self-service access to their shipment documentation and communication, without giving them access to the internal ERP.

Main Features: account activation/login (separate auth flow), document checklist responses, document upload/versioning with a confirmation step, shipment-linked document views, query threads with staff, ratings, and notification/preference management.

Database: `CustomerPortalUser` / `Invitation` / `Session` / `AuditLog` / `Notification` , `CustomerDocumentSubmission` / `Version` , `CustomerChecklistResponse` , `CustomerQueryThread` / `Message` .

API: 17 route groups under `/api/customer-portal/*` .

Current Implementation Status: Implemented, with its own security posture (separate rate-limited login flow, audit log). A dedicated `portal-placeholder.tsx` component exists and is used where portal UI is not yet fully built out for a given screen — a genuine, named placeholder rather than an inferred gap. Only 3 automated test files.

Evidence: `src/modules/customer-portal/*` ; `src/app/api/customer-portal/*` ; `prisma/schema.prisma` .

Module 10 — Recruitment (ATS + Job-Seeker Portal)

Purpose: Internal applicant tracking plus a public-facing job-seeker experience.

Main Features: job openings, candidates, applications with stage tracking, interviews, offers (internal ATS side); and a job-seeker-facing side with saved jobs, AI-assisted job matching, AI-tailored resume generation, and an AI "career conversation" assistant.

Feature flag: the entire module is gated behind `RECRUIT_MODULE_ENABLED` (default-on unless explicitly set to `"false"`, `src/lib/recruit-flag.ts`) — this is the one clearly identified, deliberate module-level feature flag in the codebase.

Current Implementation Status: Implemented, 15 API route groups. Zero automated test files.

Evidence: `src/lib/recruit-flag.ts`; `src/app/api/recruit/*`; `prisma/schema.prisma` (Recruit* models).

Module 11 — Platform Administration

Purpose: Configure the platform itself — roles/permissions, organizational structure, sessions, security, and the design system.

Main Features: role/permission management, organizational branch/department/division management, active session management, passkey management, a live design-token/theme editor, a data-tools area, and an "organization simulation" tool. A dedicated "dev console" (performance tracker/profiler) is embedded in the app shell for internal engineering use.

Current Implementation Status: Implemented.

Evidence: `src/app/(dashboard)/admin/*`; `src/modules/admin/*`; `src/modules/core/components/monolith-app-shell.tsx`.

Module 12 — Internal AI Assistant ("Mona")

Purpose: An in-app AI assistant, backed by Google Gemini, aware of the user's RBAC context.

Main Features: chat interface, server-persisted model preference/switching, quota-cooldown handling. Also exposed to the mobile app (`/api/mobile/mona/chat`).

Current Implementation Status: Implemented. Zero dedicated automated test files.

Evidence: `src/modules/mona/gemini-client.ts`; `src/app/api/mona/*`.

Module 13 — To-Do / Personal Productivity

Purpose: Personal task management for any user.

Main Features: tasks with subtasks, reminders (upcoming/check endpoints), and — per recent commit history — an overall task progress bar.

Current Implementation Status: Implemented and fully wired (CRUD + reminders, `src/modules/todo/service.ts`, `/api/todos/*`) — one of the simpler, cleanly finished modules in the system.

Evidence: `src/modules/todo/*`; `src/app/api/todos/*`.

Module 14 — Notifications

Purpose: Cross-module notification delivery and lifecycle (acknowledge, dismiss, read, resend).

Current Implementation Status: Implemented, 8 API route groups covering the full read/dismiss/acknowledge lifecycle. Zero dedicated automated test files.

Evidence: `src/app/api/notifications/*`.

Modules Present but Not Deeply Investigated in This Pass

The following modules were confirmed to exist with real pages/routes but were not inspected to the same depth; each carries **zero automated test files**, which is noted as a coverage observation, not a functionality judgment:

- **Expense** — a single top-level page at the app-router level, with deeper logic living in `src/modules/expense`.
- **Items / Product Catalogue** — product/item master data.
- **People** — appears to be an early-stage module; a test file within it (`people-workspace.test.tsx:50`) explicitly asserts a "coming soon" stub state, suggesting this module is among the least mature in the system. **This could not be further confirmed from the current codebase** without deeper inspection.
- **Incentives** — sales incentive tracking, cross-referenced from CRM.

---

10. End-to-End Workflows

Workflow 1 — CHA Job Lifecycle (Enquiry to Billing)

Trigger: a customer enquiry is created in CRM (or a lead converts to a service enquiry). **Participants:** CRM/sales staff, CHA operations staff, the customer (via Customer Portal), accounting staff. **Modules involved:** CRM → CHA → Communication (auto-provisioned Drive/Chat) → Accounting.

Step-by-Step Flow: 1. A CRM service enquiry (customs clearance or freight forwarding type) is created and, once qualified, converted into a CHA job. 2. Job creation triggers automatic provisioning of a dedicated Google Drive folder tree and Google Chat space (`workspace-provisioning.ts`), inviting the assigned staff. 3. Required documents are defined via `ChaDocumentDefinition`, tracked through a checklist (`ChaChecklist` with section/item/approval/rework states); the customer can upload/confirm documents directly through the Customer Portal. 4. Filing data is entered against the job (import/export headers, container/invoice/item detail), validated against statutory reference data (tariff/cess/RODTEP/ROSCTL/scheme codes). 5. Job-related expenses are logged (`ChaExpenseRequest`) and can trigger customer advance receipts. 6. On completion, billing data flows into Accounting as sales documents/invoices. 7. Communication generated along the way (emails, chat messages, uploaded files) can be explicitly linked back to the job via the Mail module's job-linking feature.

Data Movement: CRM enquiry → `ChaJob` → `ChaChecklist` / `ChaFiling*` → `AccountingDocument`/invoice. **Failure Paths:** document checklist rework loop (rejected documents cycle back to the customer); due-date warning system flags jobs at risk of missing statutory deadlines. **Result:** a fully documented, billed, and filed customs clearance job with an audit trail across CRM, CHA, and Accounting.

```
flowchart LR
    A[CRM Enquiry] --> B[Convert to CHA Job]
    B --> C[Auto-provision Drive + Chat Space]
    B --> D[Document Checklist]
    D -->|Customer Upload via Portal| D
    D -->|Approved| E[Filing Data Entry]
    E --> F[Statutory Filing]
    B --> G[Job Expenses]
    F --> H[Billing → Accounting]
    G --> H
```

Workflow 2 — Employee Leave Request

Trigger: an employee submits a leave request. **Participants:** requesting employee, approving manager(s) (possibly via delegation), HR. **Modules involved:** Attendance/Leave → Notifications → (optionally) external calendar sync.

Step-by-Step Flow: 1. Employee submits a `LeaveRequest` against a `LeaveType`, checked against current `LeaveBalance` and applicable `LeavePolicyVersion` (including sandwich-rule/holiday-escaping logic). 2. Request routes through configured `LeaveApprovalStep`s; if the approver has delegated authority, it routes to the delegate. 3. On approval, a `LeaveLedgerEntry` is posted (Decimal-precision, per the float→Decimal migration done in the closure passes), balances update, and a notification is sent. 4. If configured, the approved leave syncs outward to an external calendar. 5. Compliance/reconciliation reporting reflects the change.

Failure Paths: an explicitly-fixed self-approval bypass (an employee approving their own leave) was closed in a prior security pass; IDOR protections prevent viewing/acting on another employee's request via ID manipulation. **Result:** an approved (or rejected) leave request reflected in balances, ledger, calendar, and reporting.

Note: as documented in §9 Module 6, the underlying database migration for this engine has a recorded deployment blocker (write-denied credential against the live database) — this workflow's *code* is complete and tested, but its live database state should be confirmed before relying on it in production.

Workflow 3 — Internal Communication Around a Job

Trigger: any staff member emails, chats, or shares a file related to an active CHA job. **Modules involved:** Communication (Mail/Chat/Drive) → CHA.

Step-by-Step Flow: 1. Email is sent/received in the integrated Gmail client; the user can link the thread to a job under one of 8 document categories. 2. Chat messages in the job's auto-provisioned Google Chat space are visible in-app with live presence/typing/read-receipts. 3. Files are stored in the job's provisioned Drive folder and browsable from within the job's ERP record. 4. All three feed the Communication "overview" dashboard's activity aggregation.

Result: a job's full communication trail is retrievable from within its ERP record rather than scattered across separate tools.

---

11. Data Architecture

The schema contains **415 models and 25 enums** in a single PostgreSQL database via Prisma. Core entity groups:

- **Organization/Access:** `Organisation` (tenant root) → `Branch` / `Department` / `Division`; `User` → `UserRole` → `RolePermission` → `Permission`; `UserSession`, `SecurityEvent`, `PasskeyResetRequest`.
- **HR/Attendance/Leave:** employment records, shifts, biometric punch data, the full leave engine (types/balances/requests/policy versions/ledger/approval steps/comp-off/grants), holidays, overtime, HR cases and letters, travel.
- **Performance/Learning:** appraisal cycles and their review chain, goals/skills, courses/enrollments, surveys.
- **CRM:** leads/contacts/accounts/deals/activities/tickets, invoices, call-center recording/transcript/review chain, external lead-source snapshots.
- **Accounting:** the largest domain (80+ models) — legal entities, periods, tax, banking, recurring schedules, fixed assets/depreciation, budgets, a generic document/line model, and migration tooling; **plus a parallel legacy model set** still present (see Module 1).
- **CHA/Shipping:** jobs, document definitions/checklists, import/export filing headers and statutory reference data, expense requests, customer advances; plus a generic filing-workflow (BPM-style) engine.
- **Communication:** Google Workspace connection/settings, Chat space/subscription/delivery/interaction-event tracking, typing/presence state, communication audit events, job-workspace profiles.

- **Customer Portal:** portal users/sessions/invitations/audit logs, document submissions/versions, checklist responses, query threads.
- **Recruitment:** job openings/candidates/applications/interviews/offers, plus job-seeker profile/saved-jobs/matching/tailored-resume/career-conversation tables.
- **Mobile/Field:** device registration, face enrollment, location tracking sessions/points, tracking alerts, fuel reimbursement.
- **Cross-cutting:** to-do tasks/subtasks, notifications, an email queue, generic document/file-asset storage, user agreement acceptance tracking.

Status/lifecycle is expressed almost entirely through dedicated enum types (e.g., `AccountingPeriodStatus`, `AccountingPostingAttemptStatus`, `ChaCustomsFilingDirection`, `ChaCustomsExternalSubmissionStatus`, `CrmServiceEnquiryStatus`) rather than free-text status fields — a sign of a reasonably disciplined schema design.

```
erDiagram
    ORGANISATION ||--o{ BRANCH : has
    ORGANISATION ||--o{ USER : employs
    USER ||--o{ USER_ROLE : assigned
    USER_ROLE }o--|| ROLE : defines
    ROLE ||--o{ ROLE_PERMISSION : grants
    CRM_LEAD ||--o| CHA_JOB : "converts to"
    CHA_JOB ||--o{ CHA_CHECKLIST : requires
    CHA_JOB ||--o{ CHA_EXPENSE_REQUEST : incurs
    CHA_JOB ||--o| ACCOUNTING_DOCUMENT : bills
    USER ||--o{ LEAVE_REQUEST : submits
    LEAVE_REQUEST ||--o{ LEAVE_APPROVAL_STEP : routes_through
    LEAVE_REQUEST ||--o{ LEAVE_LEDGER_ENTRY : posts
    CHA_JOB ||--o| GOOGLE_CHAT_SPACE : "auto-provisions"
    CUSTOMER_PORTAL_USER ||--o{ CUSTOMER_DOCUMENT_SUBMISSION : uploads
```

---

12. API Architecture

Style: REST-shaped Next.js Route Handlers, organized by domain under `src/app/api/<domain>/...`. No API versioning scheme was found (no `/v1/` segment convention).

Route organization: 26 top-level domains, ~230 endpoint files. The five largest domains are HRMS (33), Communication (24), Leave (21), Recruit (15) and AMS (16).

Authentication: predominantly an inline session check per route (`const session = await auth(); if (!session?.user) return 401`), a shared `getSessionOrUnauth()` helper in some routes, a shared-secret header (`requireCronSecret()`) for the 10 scheduled `/api/cron/*` endpoints, and a bearer-token scheme (`getMobileUser()`) for the 12 `/api/mobile/*` endpoints (reusing the same underlying session table via a different transport). The Customer Portal has its own separate, non-NextAuth login/session flow. As noted in §8, there is no single global gate — this is a structural pattern to be aware of, not a confirmed exploit.

Request validation: Zod schemas are used in multiple routes (confirmed in the AMS self-form-template route and elsewhere) though a system-wide validation convention was not independently confirmed for all 230 routes.

Response patterns: JSON responses via `NextResponse.json`; consistent `{error: "..."}` shape observed on auth failures across sampled routes.

Error handling: an `apiError()` helper converts thrown `ForbiddenError` (from RBAC permission checks) into structured 403 JSON responses in RBAC-gated routes.

Endpoint summary by module: see the per-module tables in §9 and the domain list in §6/§7.

13. External Integrations

Integration	Purpose	Auth Mechanism	Data Exchanged	Trigger	Status
Google Gmail	In-app email client, job-linked email	OAuth2 (Workspace scopes, per-user)	Threads, messages, attachments, labels, drafts	User action in Mail UI	**Implemented**
Google Chat	In-app real-time messaging, job-linked spaces	OAuth2 + bot service account	Messages, spaces, presence, read-state, typing	User action / auto-provisioning on job creation	**Implemented**
Google Drive	Job document storage/browsing	OAuth2 + bot service account	Files, folders	Auto-provisioning on job creation; user browsing	**Implemented**
Google Calendar	Meeting scheduling with Meet links	OAuth2 (Server Action)	Events, attendees	User action in Meetings UI	**Implemented**
Google Gemini	"Mona" AI assistant	API key	Chat prompts/responses, RBAC context	User chat interaction	**Implemented**
eSSL Biometric Devices	Attendance punch sync	Direct SQL Server connection (`mssql` driver)	Punch events	Scheduled/manual sync	**Implemented**
GST Public Portal	GSTIN lookup	Public service call (some configured credentials in env)	GSTIN registration details	On-demand lookup (likely CHA/customer onboarding)	**Implemented**, usage context not fully traced in this pass
OpenAI / Groq Whisper	Call-recording transcription (with mock fallback)	API key	Audio → transcript, Tamil/Hindi/English support	CRM call-recording ingestion	**Implemented**, explicit multi-provider fallback chain including a "Mock generator" for when no provider is configured
Justdial	External lead import	Headless browser session (cookie file)	Lead records	Scheduled cron job (`cron/justdial-import`) + manual live pull	**Implemented**

All Google Workspace OAuth tokens are encrypted at rest (AES-256-GCM) with refresh-token rotation (`src/lib/workspace-oauth.ts`).

14. Background Processing / Automation

No dedicated message queue or worker process was found. Asynchronous/scheduled work is implemented as **10 scheduled API endpoints** under `/api/cron/*` , each protected by a shared-secret header rather than a user session, presumably invoked by an external scheduler (not confirmed from the repository — see Documentation Confidence): `appraisal-trigger` , `cha-filing-query-reminders` , `crm-reminders` , `email-flush` , `google-chat-retry` , `justdial-import` , `leave-accrual` , `leave-approval-reminders` , `leave-expiry` , `todo-reminders` , `tracking-alerts` .

The leave-accrual job is confirmed idempotent (guarded by a unique key on `LeaveScheduleRun`). An `EmailQueue` model exists in the schema, paired with the `cron/email-flush` endpoint, implying a simple database-backed queue rather than a dedicated queueing technology (Redis/SQS/etc. were not found in the dependency list).

What was not found: a background job runner (e.g., BullMQ, a separate worker container), pub/sub, or event-bus infrastructure. This should not be read as a deficiency — for a Next.js monolith of this shape, cron-triggered endpoints plus a DB-backed queue is a reasonably common and adequate pattern — but it is worth the client knowing that scaling background work currently depends on whatever external scheduler triggers these cron endpoints, which **this repository does not itself define**.

---

15. Configuration and Environment

Environment variables (names only) fall into these groups, based on `.env.staging.example` and live `process.env.` usage:

- **Database:** `DATABASE_URL`, connection-pool tuning, staging-specific database credentials.
- **Auth/session:** `AUTH_SECRET`, `AUTH_GOOGLE_ID` / `SECRET`, `CRON_SECRET`, session/cookie configuration, face-data and Google-token encryption keys.
- **Email:** provider selection (`EMAIL_PROVIDER`), `RESEND_API_KEY`, SMTP fallback credentials.
- **Google Workspace/Chat:** service-account credentials, domain, delete-authorization mode, shared Drive/root folder IDs, Chat link-token secret/TTL.
- **AI/external services:** `GEMINI_API_KEY`, `OPENAI_API_KEY` / `GROQ_API_KEY`, CHA job-report AI model selection.
- **Biometric device:** eSSL SQL Server connection details.
- **GST portal:** base URL, auth path, client credentials, public key/token.
- **Storage:** upload-root paths for customer-portal documents and accounting bank-statement imports.
- **Feature flags:** `RECRUIT_MODULE_ENABLED`, `NEXT_PUBLIC_ENABLE_DEMO_FILL`, `ENABLE_SYSTEM_CLOCK`, `APP_EDITION` (a distinct "CHA Production Edition" build mode is referenced in the README). These are implemented as scattered `process.env` checks per module — **no central feature-flag service exists**.
- **Deployment/runtime detection:** Vercel and AWS Lambda environment markers, used to adjust database connection pooling behavior for serverless contexts.

No secrets, API keys, or credential values were reproduced in this document, in line with the security requirement governing this analysis. **Note:** the presence of a committed `.env` file (distinct from the `.env.staging.example` template) in the repository's working tree was observed in the environment listing at the start of this engagement; this should be reviewed by the client's team to confirm it does not contain live secrets and, if it does, that it is properly gitignored rather than tracked.

---

16. Logging, Monitoring and Error Handling

- **Application/audit logging:** a `CommunicationAuditEvent` model and a `CustomerPortalAuditLog` model provide domain-specific audit trails; a `SecurityEvent` model captures security-relevant events at the platform level.
- **Health checks:** a public, unauthenticated `/api/health` liveness endpoint exists and is wired into the Docker healthcheck configuration.
- **Performance instrumentation:** `src/lib/performance.ts` and `src/lib/request-performance.ts`, plus an in-app "dev console" (performance tracker/profiler) embedded in the application shell for internal engineering use, and dedicated performance-benchmark scripts for accounting phases and API routes.
- **Error handling:** a shared `apiError()` helper standardizes RBAC-forbidden responses; individual routes handle their own try/catch and error JSON shape (no single global error-boundary/interceptor for the API layer was found).
- **Missing/not confirmed:** no external error-tracking service (e.g., Sentry) integration was found in dependencies. No centralized application log aggregation was found. **This could not be confirmed from the current codebase** as

either present-but-unconfigured or genuinely absent — the repository does not preclude an externally configured logging pipeline outside its own code.

---

17. Testing and Quality Coverage

The repository has three Vitest configurations (`vitest.config.ts` — the default suite, which requires a live staging database and refuses to run without `.env.staging.local` ; `vitest.unit.config.ts` — pure unit tests with no database dependency; `vitest.ui.config.ts` — component-level tests) plus Playwright for performance/security and leave-specific end-to-end testing.

94 test files exist under `src/` . Distribution is highly uneven:

Module	Test files
Accounting	27
Leave	19
HRMS	8
CHA	8
Customer Portal	3
CRM	2
Communication	2
Auth, People, Performance	1 each
Admin, AMS, Attendance, Core, Dashboard, Expense, Freight-Forwarding, Google-Chat, Incentives, Items, Mona, Notifications, Recruit, To-Do	**0**

Fourteen of twenty-five top-level functional modules carry no module-local automated test coverage at all — notably including AMS and Freight-Forwarding, both of which also carry some of the highest UI-design-system non-compliance counts in the migration audit (§21), compounding the risk of undetected regressions in those areas.

Linting/type-checking: ESLint (with custom rules enforcing the canonical component boundaries and forbidding hardcoded colors) and TypeScript strict mode are configured. **No CI pipeline runs these automatically** — the ESLint config file contains an explicit comment stating "no CI runs lint in this repo today," and no `.github/workflows` directory exists. A `npm run quality` script chains structure audit → design-system verification → lint → test → build, but this is a manual, locally-invoked gate, not an enforced one.

No percentage test-coverage figure is claimed here, as no coverage report was found in the repository.

---

18. Deployment Architecture

Build: `npm run build` runs Prisma client generation followed by `next build` .

Containerization: a multi-stage Dockerfile (`deps` → `builder` [runs `prisma generate` + `next build`] → `migrator` [runs only the accounting schema deployment] → `runner` [standalone Next.js output, non-root user, `/api/health` healthcheck]). `docker-compose.yml` defines three services: `postgres` , an `accounting-migrate` service that must complete before `app` starts (itself gated by two additional env flags — `ACCOUNTING_PROVIDER_MODE` / `ACCOUNTING_PHASE6_EXECUTION` — both defaulting to `disabled` , functioning as an explicit safety kill-switch on the accounting migration step), and `app` .

Alternative deployment path: a `.vercelignore` file (excluding the Android app directory) suggests Vercel as a deployment target for the web application, separate from the Docker/self-hosted path; `next.config.ts` only enables

Next.js's `standalone` output mode when a `DOCKER_BUILD` flag is set, implying the Vercel path uses Next's default (non-standalone) build output.

CI/CD: no GitHub Actions workflow or other CI configuration was found in the repository (only a pull-request template exists under `.github/`). Build, lint, and test are run manually / locally per the `npm run quality` script.

Staging environment: a dedicated, guarded staging tooling layer exists — a marker-verified staging database, a PowerShell helper script (`staging-db.ps1`), and test-runner wrappers that refuse to execute the default test suite without a verified `.env.staging.local` file present.

Production deployment cannot be conclusively established from the repository — no infrastructure-as-code, cloud account configuration, or production runtime manifest was found. Whether the system runs on Vercel, the Docker path, or elsewhere in production **could not be confirmed from the current codebase**.

---

19. Current System Capabilities

Capability	Available Now	Notes
CHA job management & statutory filing	Yes	Mature, deepest module by page/route count alongside accounting
Double-entry accounting & financial reporting	Yes	Functional, but running on a dual old/new data model mid-migration
CRM (leads → deals → quotes → tickets)	Yes	Two specific quote features explicitly deferred pending a schema change
Freight forwarding documentation (HBL/MBL)	Yes	No automated test coverage
HR records, letters, travel, HR helpdesk	Yes	Onboarding *checklist UX* specifically is a stub
Attendance & biometric sync	Yes	Timesheets view is a stub
Leave management	Partial	Code/tests complete; live database migration application is an unresolved, documented blocker
Performance appraisals & learning	Yes	No automated tests
Internal email (Gmail-integrated)	Yes	Production-shaped, real Google API
Internal chat (Google Chat-integrated)	Yes	Functionally complete; UI still flagged non-compliant with new design system
Shared drive (per job)	Yes	Explicit fallback state when a folder isn't yet provisioned
Meetings/calendar	Yes	Real Google Calendar + Meet integration
Customer self-service portal	Yes	Some screens use a named placeholder component where UI isn't fully built
Recruitment / job-seeker portal (incl. AI resume/matching)	Yes	Behind a feature flag, default-on
In-app AI assistant ("Mona")	Yes	No automated tests
Personal to-do list	Yes	Cleanly and fully implemented
Platform administration (roles, org structure, design tokens)	Yes	—
Multi-factor authentication	No	Hook exists in code but is a hardcoded no-op
Automated CI/CD pipeline	No	No CI configuration found; quality gate is manual
Centralized/global API authorization gate	No	Per-route enforcement only, no framework-level backstop

---

20. Completed Work

Grouped by module, based on the evidence above:

- **Accounting:** invoicing, journals, ledgers, financial statements, bank reconciliation, fixed assets/depreciation, tax settlement, report builder, period locking, and a formally phase-gated migration tooling layer — the most thoroughly built and tested module.
- **CHA:** full job lifecycle, document checklists with approval/rework, statutory filing data entry, job-linked expenses, customer advances, due-date warnings, plus a generic filing-workflow engine underneath.
- **Freight Forwarding:** booking, HBL/MBL document creation.
- **CRM:** lead-to-deal-to-quote pipeline, service enquiries, tickets, call-center recording/transcription/QA, automated external lead import.
- **HRMS:** employee records, letters, travel, helpdesk, a two-sided recruitment system.
- **Leave Management:** the most rigorously closed-out module — full policy/approval/ledger/comp-off/delegation engine, security-hardened across 22 iterative passes, well tested.
- **Attendance:** punch tracking, biometric sync, shifts, overtime, regularization.
- **Performance/Learning:** appraisal cycles with full review chain, course/enrollment tracking.
- **Communication:** production-grade Gmail, Google Chat (with live presence/typing/read-receipts), Drive, and Calendar integration, plus automatic per-job workspace provisioning.
- **Customer Portal:** external login, document submission/versioning, checklists, query threads.
- **Platform:** RBAC, org structure management, live design-token editing, session/passkey administration.
- **Utility modules:** To-Do (fully wired), Notifications (full lifecycle), Mona AI assistant.

---

21. Historical / Previous Functionality

- **Pre-Monolith legacy UI:** git history and a dedicated verification script (`scripts/verify-old-ui-backup.mjs`) confirm an entire prior UI implementation existed before the current "Monolith" design system, checksummed and archived to an `OLD UI code/` directory (not present in the current working tree, but referenced extensively as load-bearing backup evidence throughout `docs/ui-migration-status.md` and `docs/ui-migration-handoff.md`). Commit `7120d79` ("checkpoint: working dashboard before full UI migration") marks the snapshot point before the rewrite began at commit `aed95fe` ("establish Monolith UI migration foundation").
- **Frappe-based predecessor:** a `frappe_docker/design` directory and root-level `frappe-ui-design-system.css` / `.md` files indicate the system was, at an earlier stage, based on or modeled after Frappe (an open-source ERP framework) before evolving into the current bespoke Monolith design system. **The extent to which Frappe itself was ever the runtime platform, versus only a design reference, could not be confirmed from the current codebase.**
- **Two independent, pre-existing leave systems:** `docs/leave-management/FINAL_REPORT.md` documents that, prior to the current Leave Management module, two independent and non-reconciled leave surfaces existed within `src/modules/attendance/service.ts` and `src/modules/hrms/service.ts` ("Surface A" and "Surface B"). These are now wrapped/superseded by the new `src/modules/leave/` module, though the compatibility wrapper layer remains present in the codebase.
- **A dual accounting model:** as detailed in Module 1, an older, non- `Accounting` -prefixed set of financial tables (`Account` , `JournalEntry` , `SalesInvoice` , etc.) remains in the schema alongside the new `Accounting*` model set — direct, current evidence of a still-incomplete rewrite rather than a purely historical artifact.
- **A removed filing-phases feature:** commit `8d65f6c` ("remove customs filing and continue ui migration") shows a customs-filing-phases feature that was built and then explicitly removed/descoped — the specifics of what was removed and why **could not be fully reconstructed from the commit message alone**.
- **Early-stage appraisal forms:** two of the earliest commits in the repository (`9852081` , `aa9915d`) explicitly marked appraisal/reviewer forms as "(Not Completed)" during initial scaffolding — historical context only; no equivalent live incompleteness marker was found in the current AMS module code during this pass.

22. Pending and Incomplete Work

Area	Pending Item	Evidence	Impact	Status
Attendance	Timesheets view is a stub	`attendance/timesheets/page.tsx:13` — "Timesheets coming soon"	Users cannot view/manage timesheets through this route	Pending
HRMS	Onboarding checklist UX not built	`hrms/components/onboarding-view.tsx:10,16`	New-hire onboarding lacks a guided checklist experience	Pending
CHA	One settings sub-panel unbuilt	`cha/settings/settings-form.tsx:1101` — "Coming soon"	Minor — a single configuration panel	Pending
CRM	Quote sharing feature deferred	`QuoteDetailsPage.tsx:759` — "coming soon (pending a database update)"	Users cannot share quotes externally via this control yet	Pending, blocked on schema change
CRM	Quote workflow preferences deferred	`QuoteDetailsPage.tsx:792`	Same as above	Pending, blocked on schema change
Accounting	Legacy and new financial model sets coexist	`prisma/schema.prisma` — parallel `Accounting*` vs. legacy `Account` / `JournalEntry` /etc. models, plus dedicated migration-tooling tables	Ongoing complexity/risk of data being in the "wrong" model during the transition	Partially Implemented (in-progress migration)
Leave Management	Ledger/policy-engine migration not applied to live database	`docs/leave-management/FINAL_REPORT.md` — write-denied credential against target environment	Module's schema changes may not be live despite code/tests being complete	**Deployment blocker — needs operational verification**
UI / Design System	55 of 341 routes non-compliant, 33 partially compliant, with the entire audit being static-analysis-only (no route has passed runtime/browser verification)	`docs/UI_DESIGN_SYSTEM_MIGRATION_STATUS.md` (audit run 2026-08-19)	Visual inconsistency risk in non-migrated areas; unverified runtime behavior even in "compliant" routes	Partially Implemented
Architecture Governance	`architecture:check` script fails — `src/components/monolith` still holds implementation files outside its allowed barrel-only contract	`docs/ui-migration-handoff.md` (repeated, unresolved across many logged sessions)	A committed project rule is currently unenforced/failing in the actual codebase	Confirmed, unresolved
Architecture Governance	`design-system:verify` script fails — `design-system-catalogue.css` file is missing	`docs/ui-migration-handoff.md`	Same as above	Confirmed, unresolved

Area	Pending Item	Evidence	Impact	Status
Security	Multi-factor authentication is a hardcoded no-op	`src/lib/auth.ts` — `verifySecondFactor()` always returns `true`, commented as a placeholder	MFA is not actually enforced anywhere despite the code path existing	Present but Disabled
Testing	14 of 25 modules have zero automated tests	`src/**/__tests__` / `*.test.ts` inventory (94 files total, concentrated in Accounting/Leave)	Elevated regression risk in AMS, Freight-Forwarding, CRM (relative to size), Communication (relative to size), and others	Confirmed gap
CI/CD	No CI pipeline found	`eslint.config.mjs` explicit comment; no `.github/workflows`	Quality gates (lint/test/build) are not automatically enforced on changes	Confirmed gap
Module maturity	`people` module appears to have an actively-asserted stub state in its own test suite	`people-workspace.test.tsx:50`	Scope/maturity of this module unclear	Cannot Be Determined further without deeper inspection

---

23. Technical Debt and Risks

Confirmed Technical Debt

- **Dual accounting data model** (old and new table sets coexisting) — directly visible in `prisma/schema.prisma`.
- **Unresolved architecture-governance failures**: two project-mandated verification scripts (`architecture:check`, `design-system:verify`) are currently failing against the actual codebase, per the project's own migration handoff log.
- **Per-route (not centralized) authentication/authorization enforcement** across ~230 API endpoints — functionally consistent in practice via a few shared idioms, but with no framework-level guarantee that every new route will apply the check.
- **Legacy CSS compatibility layer** (`src/styles/legacy-compatibility.css`), explicitly forbidden from further extension per `AGENTS.md`, existing purely to keep older styling working during the ongoing migration.
- **Heavily uneven test coverage** (94 files, 14 of 25 modules at zero) relative to the system's size and business criticality.
- **No CI/CD pipeline** — quality gates are manual and can be skipped.

Potential Risks

- **MFA scaffolding without enforcement** could create a false sense of security if stakeholders assume the presence of the code path means MFA is active — it is not.
- **Department-scoped implicit permission bundles** (e.g., automatic accounting access based on department membership) could over-grant access relative to a strict least-privilege model; this is a design choice worth revisiting from a security-review perspective, not a confirmed vulnerability.
- **Unapplied leave-management database migration**: if the module is being treated as "done" by stakeholders without confirming the migration has actually run against production, there is a real risk of a mismatch between code

expectations and live schema.

- **A tracked `.env` file** was observed in the working directory at the start of this engagement (separate from the `.env.staging.example` template); this warrants a direct check by the client's team that no live secrets are committed to source control.
- **Static-only UI compliance audit**: the design-system migration status is entirely derived from static analysis; the project's own documentation repeatedly notes that browser/runtime verification has been blocked in the development environment, meaning even "compliant" routes have not been confirmed to render correctly at runtime through this audit process specifically (this does not mean they don't work — only that this particular verification step hasn't happened).
- **Background/cron reliability depends on an external scheduler** not defined within this repository — its configuration, monitoring, and failure-alerting could not be assessed from the codebase alone.

---

24. Recommended Next Steps

Critical

- Confirm, operationally, whether the Leave Management database migration (`20260814090000_leave_management_ledger_policy_engine`) has been applied to the live production database. If not, resolve the write-access blocker and apply it before relying on the module in production.
- Verify no live secrets are present in the tracked `.env` file; if any exist, rotate them and remove the file from version control.

High Priority

- Complete the accounting old→new model migration and retire the legacy table set, closing the current dual-model window.
- Resolve the two currently-failing architecture-governance checks (`architecture:check`, `design-system:verify`) so the project's own quality gates reflect reality.
- Add automated test coverage to the highest-risk zero-coverage modules first: AMS and Freight-Forwarding (both large, both flagged with high UI non-compliance counts), followed by CRM and Communication (large modules with disproportionately thin coverage relative to size).
- Establish a CI pipeline that runs lint, type-check, and tests automatically on every change — this is currently a fully manual step.

Medium Priority

- Complete the UI/design-system migration for the remaining non-compliant routes, and establish an actual runtime/browser verification step (rather than static analysis only) to close the gap repeatedly noted as blocked in the project's own migration logs.
- Deliver the explicitly deferred features: attendance timesheets, HR onboarding checklist UX, the one CHA settings panel, and the two CRM quote features pending their schema change.
- Review the department-scoped implicit permission-bundle mechanism against the organization's intended least-privilege posture.
- Consider centralizing API authentication/authorization enforcement (e.g., a shared middleware or route-wrapper convention enforced by lint rule or code generation) to remove reliance on every route author remembering the check.

Future Enhancements

- Activate the existing MFA hook with a real second-factor implementation, given the scaffolding already exists.
- Consider a dedicated background-job/queue technology if cron-endpoint-triggered processing outgrows its current scale, particularly for the Google Workspace retry and email-flush paths.

- Extend automated coverage to the remaining zero-coverage utility modules (To-Do, Notifications, Mona, Recruit) as they mature.

---

25. Conclusion

Monolith Engine is a substantial, actively developed, and largely production-shaped ERP system. Its core business-critical paths — CHA job processing, accounting, CRM, HR, and internal communication — are deep, functional, and in several cases (Leave Management, Accounting) demonstrate a genuinely rigorous engineering process, including security review, iterative closure of real defects, and phase-gated rollout discipline.

The system's overall maturity is best described as **mid-transition**: two large, well-documented but incomplete initiatives — a UI/design-system rewrite and an accounting data-model rewrite — are running concurrently with ongoing feature development, which is a normal but real source of temporary complexity. The most material open items for the client to track are the unresolved leave-management database-migration blocker, the dual accounting model, the two failing architecture-governance checks, and the unevenness of automated test coverage across modules.

None of these findings suggest the platform is unstable or non-functional — the evidence points to a system that works today for its core workflows, being actively improved by a team that documents its own gaps unusually thoroughly. The recommended next phase of work should prioritize closing the in-flight migrations and shoring up test coverage in the modules currently running without a safety net, ahead of adding further net-new feature scope.

---

26. Appendix — Module Status Matrix

Module	Status	Backend	Frontend	Database	Tested	Evidence
Accounting	Partial (active migration)	Yes	Yes	Yes (dual model)	Yes (27 files)	`prisma/schema.prisma`, `src/modules/accounting/*`
CHA	Complete	Yes	Yes	Yes	Yes (8 files)	`src/modules/cha/*`
Freight Forwarding	Complete	Yes	Yes	Yes	No	`src/app/(dashboard)/freight-forwarding/*`
CRM	Complete (2 deferred features)	Yes	Yes	Yes	Yes (2 files)	`src/modules/crm/*`
HRMS	Complete (1 stub view)	Yes	Yes	Yes	Yes (8 files)	`src/modules/hrms/*`
Attendance	Complete (1 stub route)	Yes	Yes	Yes	Partial (shared w/ leave)	`src/modules/attendance/*`
Leave Management	Complete (code); DB migration blocker	Yes	Yes	Yes (unapplied migration)	Yes (19 files)	`docs/leave-management/*`
AMS (Appraisals/Assets)	Complete	Yes	Yes	Yes	No	`src/app/(dashboard)/ams/*`
LMS (Learning)	Complete	Yes	Yes	Yes	No	`src/app/(dashboard)/lms/*`
Communication (Mail/Chat/Drive/Calendar)	Complete	Yes	Yes	Yes	Yes (2 files)	`src/modules/communication/*`

Module	Status	Backend	Frontend	Database	Tested	Evidence
Customer Portal	Complete (some placeholder screens)	Yes	Yes	Yes	Yes (3 files)	`src/modules/customer-portal/*`
Recruitment (ATS + Job Seeker)	Complete (feature-flagged)	Yes	Yes	Yes	No	`src/lib/recruit-flag.ts`
Admin/Platform	Complete	Yes	Yes	Yes	No	`src/app/(dashboard)/admin/*`
Mona (AI Assistant)	Complete	Yes	Yes	N/A	No	`src/modules/mona/*`
To-Do	Complete	Yes	Yes	Yes	No	`src/modules/todo/*`
Notifications	Complete	Yes	Yes	Yes	No	`src/app/api/notifications/*`
People	Cannot Determine (possible stub)	Unclear	Unclear	Unclear	Test asserts "coming soon"	`people-workspace.test.tsx:50`
Expense	Complete (thin at route level)	Yes	Yes	Yes	No	`src/modules/expense/*`
Incentives	Complete	Yes	Yes	Yes	No	`src/app/(dashboard)/crm/incentives`

---

27. Appendix — Source Reference Index

Topic	Primary Source Files
Authentication	`src/lib/auth.ts`, `src/lib/session-service.ts`, `src/lib/session-config.ts`, `src/lib/login-rate-limit.ts`
Authorization / RBAC	`src/lib/rbac.ts`, `src/lib/api-helpers.ts`
Mobile auth	`src/lib/mobile-auth.ts`
Database / ORM	`prisma/schema.prisma`, `src/generated/prisma`, `src/lib/db.ts`
Accounting logic	`src/modules/accounting/*`, `src/modules/accounting/posting-engine.ts`
CHA logic	`src/modules/cha/*`
Communication / Google integrations	`src/lib/google-chat-client.ts`, `src/lib/google-gmail-client.ts`, `src/lib/google-calendar-client.ts`, `src/lib/google-drive-client.ts`, `src/lib/workspace-oauth.ts`, `src/lib/workspace-provisioning.ts`
AI assistant (Mona)	`src/modules/mona/gemini-client.ts`
Biometric device integration	`src/lib/essl.ts`
GST integration	`src/lib/gst-public-search.ts`
Call transcription	`src/lib/transcription.ts`
Leave management	`src/modules/leave/*`, `docs/leave-management/*`
UI / design system migration	`docs/UI_DESIGN_SYSTEM_MIGRATION_STATUS.md`, `docs/ui-migration-status.md`, `docs/ui-migration-handoff.md`, `docs/ui-route-audit.md`, `docs/ui-component-and-style-ownership-audit.md`, `AGENTS.md`

Topic	Primary Source Files
Legacy UI backup	`scripts/verify-old-ui-backup.mjs`, `scripts/migrate-component-architecture.mjs`, `scripts/split-monolith-module-styles.mjs`
Deployment	`Dockerfile`, `docker-compose.yml`, `.vercelignore`, `next.config.ts`, `package.json`
Testing	`vitest.config.ts`, `vitest.unit.config.ts`, `vitest.ui.config.ts`, `scripts/leave-playwright-e2e.cts`

Documentation Confidence and Limitations

This document was produced through direct inspection of the source code, database schema, configuration files, scripts, project documentation, and 309 recorded git commits. Confidence is high for structural claims (module inventory, API surface, database schema, technology stack) since these were verified against the actual repository content rather than inferred from naming alone.

Confidence is lower, or explicitly unconfirmed, in the following areas:

- **Production runtime environment:** whether the system currently runs on Vercel, the Docker path, or another target in production could not be established from the repository alone — no infrastructure-as-code or live deployment manifest was found.
- **External scheduler for cron endpoints:** the `/api/cron/*` endpoints assume an external trigger (a scheduler service), which is not itself defined in this repository.
- **Live database state for Leave Management:** the codebase and its own documentation flag an unresolved migration-application blocker; this document reports that blocker as documented but could not independently verify the current live database state.
- **Runtime behavior of the UI migration:** the design-system compliance audit referenced throughout this document is static-analysis-only; the project's own logs repeatedly note that browser-based runtime verification has been unavailable in the development environment used, so "compliant" status per that audit reflects source-level conventions, not confirmed visual/runtime correctness.
- **Mobile application (`mobile/sales-crm-android`):** referenced and cross-linked from the web API (`/api/mobile/*`) but not inspected in source-code depth during this pass.
- **Full extent of the Frappe-based predecessor system:** residual files reference an earlier Frappe-based design; whether Frappe was ever the actual runtime platform (versus a design reference only) could not be confirmed from the current codebase.
- **`people` module maturity:** a test file suggests a "coming soon" stub state, but the module's true current scope could not be fully determined in this pass.
- **External monitoring/error-tracking:** no such integration was found in the repository's dependencies, but this does not rule out an externally configured pipeline outside the scope of this codebase.

Where this document states a claim as fact, it is grounded in the cited source file(s). Where evidence was incomplete, this document has said so explicitly rather than inferring a conclusion.